SIMATS ENGINEERING

ASSESSMENT TOOL 3

Course Name	Artificial Intelligence	Course Code	CSA17
Name	Krise Monali	Register Number	192519111

Course Outcome Covered: CO2 – Search and game-solving techniques (Greedy Search, A*, CSP, Minimax, Alpha-Beta pruning) with heuristic functions (BL3)

Annexure A – Dry Run Evaluation

Code of Conduct: I, Krise Monali (Reg No: 192519111), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Question 1 – Breadth-First Search (BFS)

Graph: A is the root with children B and C; B has children D and E; C has children F and G. BFS uses a FIFO **queue**. The queue column shows the contents after the visited node is dequeued and its children are added.

Iteration	Queue	Visited Node
1	[B, C]	A
2	[C, D, E]	B
3	[D, E, F, G]	C
4	[E, F, G]	D
5	[F, G]	E
6	[G]	F
7	[ ]	G

Questions

1. **BFS traversal order:** A → B → C → D → E → F → G (level by level: A | B, C | D, E, F, G).
2. **Queue contents after each iteration:** [B, C], [C, D, E], [D, E, F, G], [E, F, G], [F, G], [G], [ ] (as shown in the table; the queue starts as [A]).

Question 2 – Depth-First Search (DFS)

DFS uses a **stack** (LIFO). The top of the stack is written on the left. Children are pushed right-to-left so that the left child is explored first.

Iteration	Stack	Visited Node
1	[B, C]	A
2	[D, E, C]	B
3	[E, C]	D
4	[C]	E
5	[F, G]	C
6	[G]	F
7	[ ]	G

DFS traversal order: A → B → D → E → C → F → G. The stack starts as [A]; D and E are leaves, so DFS backtracks to C and then explores F and G.

Question 3 – 8-Puzzle using BFS

Initial state (blank = 0): 1 3 6 / 5 0 2 / 4 7 8. Goal state: 1 2 3 / 4 5 6 / 7 8 0.

Notation and assumptions: a state is written row by row, e.g. 136/502/478 means rows (1 3 6), (5 0 2), (4 7 8). Moves are the blank moving Up, Down, Left, Right (in that order). BFS uses a FIFO queue, and states already generated are not added again. States are named S0, S1, ... in the order they are generated.

State	Configuration	Generated from
S0	136/502/478	Initial state
S1	106/532/478	S0 – Up
S2	136/572/408	S0 – Down
S3	136/052/478	S0 – Left
S4	136/520/478	S0 – Right
S5	016/532/478	S1 – Left
S6	160/532/478	S1 – Right
S7	136/572/048	S2 – Left
S8	136/572/480	S2 – Right
S9	036/152/478	S3 – Up
S10	136/452/078	S3 – Down
S11	130/526/478	S4 – Up
S12	136/528/470	S4 – Down

Trace table (first 5 iterations)

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	S0 (136/502/478)	[S0]	[S1, S2, S3, S4]
2	S1 (106/532/478)	[S1, S2, S3, S4]	[S2, S3, S4, S5, S6]
3	S2 (136/572/408)	[S2, S3, S4, S5, S6]	[S3, S4, S5, S6, S7, S8]
4	S3 (136/052/478)	[S3, S4, S5, S6, S7, S8]	[S4, S5, S6, S7, S8, S9, S10]
5	S4 (136/520/478)	[S4, S5, S6, S7, S8, S9, S10]	[S5, S6, S7, S8, S9, S10, S11, S12]

Note: the goal is at depth 8, so BFS cannot reach it in 5 iterations. The five rows above are the first five iterations of the trace; BFS then continues in the same way, level by level (the 4 states at depth 1, then depth 2, and so on) until the goal is removed from the queue.

Questions

1. **Which node is expanded in each iteration?** Iteration 1: S0 (the initial state); 2: S1; 3: S2; 4: S3; 5: S4. The search continues expanding in FIFO order (S5, S6, S7, ...) until the goal is reached at depth 8.
2. **How many states are generated in total?** After the 5 iterations shown, 13 states have been generated (S0 plus 12 new states: 4 + 2 + 2 + 2 + 2). For the complete search, BFS expands 310 states and has generated **500 states in total** (including the initial state) when the goal state is taken from the queue.
3. **Complete solution path** (8 moves, path cost 8):

Step	Move of blank	State
0	Initial	1 3 6 / 5 0 2 / 4 7 8
1	Right	1 3 6 / 5 2 0 / 4 7 8
2	Up	1 3 0 / 5 2 6 / 4 7 8
3	Left	1 0 3 / 5 2 6 / 4 7 8
4	Down	1 2 3 / 5 0 6 / 4 7 8
5	Left	1 2 3 / 0 5 6 / 4 7 8
6	Down	1 2 3 / 4 5 6 / 0 7 8
7	Right	1 2 3 / 4 5 6 / 7 0 8
8	Right	1 2 3 / 4 5 6 / 7 8 0 (Goal)

Path: Right → Up → Left → Down → Left → Down → Right → Right.

1 **Why does BFS always find the shortest solution in an unweighted 8-puzzle?** BFS expands nodes in FIFO order, so it expands every state at depth d before any state at depth d + 1. In the 8-puzzle every move has the same cost (1), so path cost equals depth. The first time the goal is reached, no shallower path to it can exist, because all shallower states were already expanded without finding it. The branching factor is finite (at most 4), so BFS is also complete. The price is memory and time of O(b^d), which grows quickly (500 states for a depth-8 solution here).
